# Space-time deterministic graph rewriting

Pablo Arrighi[1], Marin Costes[1], Gilles Dowek[1], and Luidnel Maignan[2]

[1] Université Paris-Saclay, Inria, CNRS, LMF, 91190 Gif-sur-Yvette, France
[2] Univ Paris Est Creteil, LACL, 94000 Creteil, France

**Abstract.** We study non-terminating graph rewriting models, whose local rules are applied non-deterministically—and yet enjoy a strong form of determinism, namely space-time determinism. For terminating computation, it is well-known that the property of confluence may ensure a deterministic end result. In the context of distributed, non-terminating computation however, confluence alone is too weak a property. Here we provide sufficient conditions so that asynchronous local rule applications produce well-determined events in the space-time unfolding of the graph, regardless of their application orders. Our first two examples are asynchronous simulations of dynamical systems. Our third example features time dilation, in the spirit of general relativity, as a prime illustration of a phenomenon that is fundamentally asynchronous yet gives rise to a consistent space-time.

**Keywords:** Causal graph dynamics · Cellular automata · Covariance · Commutation · Strong confluence · Distributed computation · Task dependencies · DAG · Poset · Space-like cut · Foliation.

## 1 Introduction

### 1.1 Context

*Dynamical systems* refer to the global and synchronous (or almost synchronous [27]) evolution of an entire configuration at time $t$ into another configuration at time $t + 1$, and then $t + 2$, etc., iteratively. Dynamical systems often have spatial dimensions too, as the configurations are often grid-based (e.g. for representing particles, fluids, traffic jams... But they can also be graph-based (e.g. representing physical systems, computer processes, biochemical agents, economical agents, users of social networks, etc.). One can think of cellular automata, lattice-gas automata, parallel graph rewriting, causal graph dynamics [1,6,3] or so-called global transformations [26] for instance.

*Synchronism* is often criticised however, on the basis that: 1/ In some contexts synchronization mechanisms are considered a costly resource, which hinders the use of parallelism to achieve high performance computing [30]. 2/ It is often dubbed as physically unrealistic. Some authors argue that nature has no central clock, and thus cannot apply the same rule everywhere at once [25]. For instance, relativistic physics departs from the idea of a global time across the universe, because its 'time covariance' symmetry makes it perfectly legitimate to evolve just a small region of space, whilst keeping the rest unchanged.

*Asynchronism*—the application of the local rule at totally arbitrary places, non-deterministically—fits this picture better and is also well studied, especially when it is

more compelling to leave the evaluation strategy under-determined. This is typically the case in rewriting theory, when the rewrite rules arise as a computationally-oriented version of an equality, e.g. $1 + 1 \to 2$. Then, term $1 + 1 + 1$ may evolve into $2 + 1$ or $1 + 2$, non-deterministically. This feels right, because: 1/ an underlying symmetry tells us there is no reason to favour one over the other and 2/ ultimately, if what matters is the end result, we are reassured by the fact that $2 + 1 \to 3$ and $1 + 2 \to 3$.

More generally, *confluence* of a set of rewrite rules is the property that if the evolutions $a \to^* b$ and $a \to^* c$ are possible, then the evolutions $b \to^* d$ and $c \to^* d$ are also possible for some $d$. In the context of terminating computation, this ensures that the non-determinism introduced by the order of application of the rewrite rules does not matter to the end result, yielding a well-determined, unique normal form, e.g. 3 in the above example. Contributions [15,23,20,10,22] deal with the case when the computational process eventually produces a graph as a result, through successive rewrites.

### 1.2 Motivation

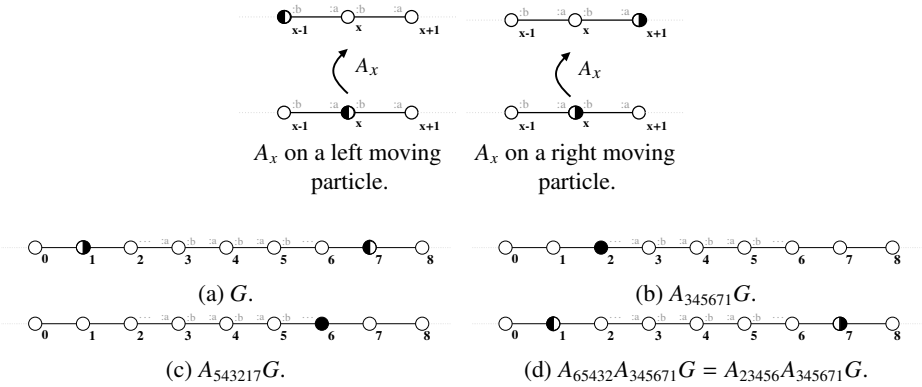



Fig. 1: *Confluence yet inconsistencies.* The local rule transports left-moving particles to the left, and right-moving particles to the right, where left/right are distinguished thanks to the $a/b$ ports. ($a$) We start with a left-moving particle in 7 and a right-moving particle in 1. ($b$)&($c$) The point of collision between the two particles is not well defined, it depends on the evaluation strategy. Here $A_{71}G$ is short for $A_7(A_1G)$. ($d$). Still, the system is confluent, as the divergent configurations can be both evolved into the last.

We are interested in computational processes that generate not just one result, but a non-terminating succession of them. Unfortunately, confluence alone hardly provides any guarantee in such cases as, for instance, rewrite rules yielding every possible result are computationally meaningless but trivially proven confluent. The use of labelling techniques [24] aiming at safeguarding the successive results from being rewritten can still save the day at this stage. But things will go out of hand if we are interested in modelling not just one computational process, but an entire network of interacting

processes, each generating results that cannot be safeguarded, because they are being consumed by the neighbouring processes to produce their next result, etc., i.e. non-terminating distributed computation over linear resources, e.g. particle systems driven by number-conserving rules. For instance, Fig. 1 shows left and right-moving particles on a line, where left versus right is indicated by the ports along the edges. The local rule simply has them move, which means consuming the particle at vertex, to give it to another. A little thought shows that the system is confluent. Yet, depending upon the chosen order of applications, one finds that one particle propagates much faster than the other, yielding the collision event to occur at very different places. This means that spacetime events ill-defined, inconsistent. Whilst this example is designed to emphasize these issues, it is clear that, when it comes to dynamical systems, asynchronous evaluation strategies may lead to inconsistent results, nonphysical effects (e.g. superluminal signalling), and pathological dynamical behaviours (e.g. over Boolean networks [31,13] and cellular automata [35,17]). This is a major deterrent to their usage, and the notion of confluence is no fix to that.

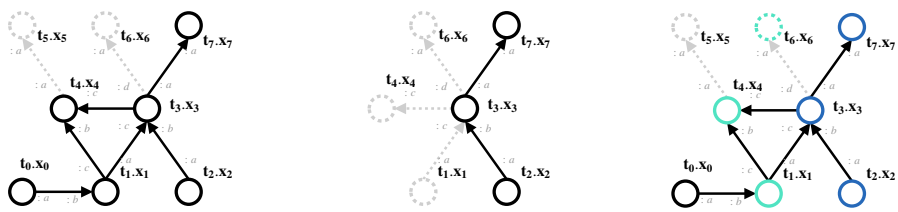
In order to fix this, we introduce the notion *space-time determinism* in two variants. In both cases, the idea is to move beyond the notion of confluence, which traditionally compares configurations globally while ignoring their spatial and local structures, and introduce a spatially and locally refined notion instead. It will ensure the existence of well-defined, interrelated events constitutive of a global "space-time diagram" (in the sense of cellular automata or Physics). The first approach, referred to as *weak space-time determinism*, focuses on maintaining consistent a certain class of events—that can be thought of as "results"—across all possible evaluation strategies. The second approach, called *full space-time determinism*, considers all events instead—they can be thought of as both "partial results" and results of the previous class. Intuitively, it asks that two snapshots of the evolving graph that have consumed the same information in some region, be consistent upon the (partial) results in this region—regardless of the evaluation strategies used to compute each snapshot. The main theorems of this paper show that some natural, local conditions upon the local rule, entail (weak or full) space-time determinism.

*Dynamical geometry.* We do not restrict ourselves to work over a fixed lattice or a fixed boolean network, here. Our processes are initially laid out in some network, but then they can connect with the neighbours of their neighbours, disconnect etc., making the network evolve. An important insight is that in order to obtain space-determinism, the network needs be directed, and understood as a DAG of dependencies between the processes. The DAG of dependencies is both a constraint upon the evolution, and a subject of the evolution, allowing us to express intriguing effects such as time dilation, reminiscent of general relativity, see Fig. 9.

*Applications.* Whilst mainly of theoretical nature, this work does suggest efficient parallel schemes for the implementation of dynamical system and distributed systems—doing away with any expensive clock synchronisation mechanisms and replacing it with just a cheap DAG. Another strand of applications lies in Physics, namely seeking for mathematically sound, constructive frameworks for discrete models of general relativity [36]. From a general philosophy of science point of view, we find it compelling to reconcile asynchronism and determinism.

*Plan of the paper.* Sec. 2 lays out the formalism for graphs and local rules. Sec. 3 provides an example of how to perform the asynchronous simulation of a particle system, at the cost of introducing "metric" information in the form of these DAGs so as to capture the relative advancement of the computation in a region with respect to another. Sec. 4 generalizes this construct to simulate any cellular automaton. Sec. 5 shows how, having introduced this DAG, one can manipulate it to achieve time dilation effects. An analogy is drawn with various concepts coming from general relativity, such as time covariance, metric, background-independence and dynamical geometry. Sec. 6 rigorously defines the weak/full space-determinism properties motivated by these examples. It introduces a set of local conditions, also met by the examples, and demonstrates how these entail weak/full space-determinism. These theorems constitute our main technical contributions, with full proofs found provided in the long version of this paper. Sec. 7 summarizes the results, compares them to the related works, and provides some perspectives.

## 2 Graphs and local rules




(a) $G$ (b) $G_{\{x_2,x_3,x_7\}}$ (c) $G$ with $X$ and $X^-$.

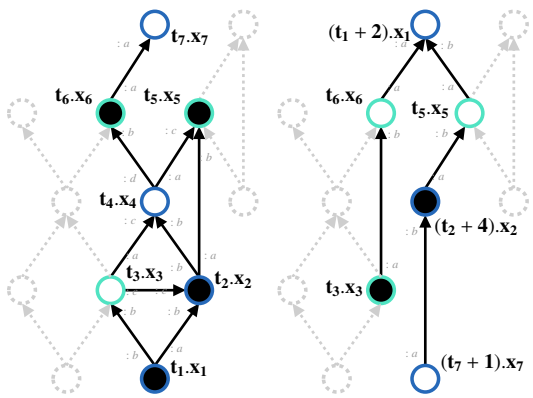
Fig. 2: *Induced subgraph and borders (a)(b):* We consider a graph $G$ and its induced subgraph $G_{\{x_2,x_3,x_7\}}$. Graphs have borders, as shown pointed by the dashed lines for $(a)$ $G$ and $(b)$ $G_{\{x_2,x_3,x_7\}}$. *Interior of a set (c):* We consider a set of positions $X = \{x_1, x_2, x_3, x_4, x_6, x_7\}$. Positions in this set are either interior (denoted $X^-$ and shown in dark blue) or in the boundary ($X \setminus X^-$ in cyan).

We start by formally introducing the type of graphs that we consider: directed acyclic labelled port graphs. The use of labelled port graphs is is totally standard in distributed computing [12] This is because ports are mandatory for expressiveness, in order to be able to tell a neighbouring process from another. So are the labels of the nodes, in order to capture the internal state of each process. The DAG structure is mandatory to capture the dependency between the processes: this is also quite common [9,30].

Each vertex $t.x$ may be understood as an event, featuring a computational process at position $x$ and time tag $t$.

**Definition 1 (Positions, ports, states, names).** *Let $\mathcal{X}$ be an infinite countable set of* positions*. Let $\pi$ be a finite set of* ports *and $\Sigma$ be a finite set of* states*. The set of* names *is defined as $\mathcal{V} := \{\, t.x \mid t \in \mathbb{Z}, x \in \mathcal{X} \,\}$.*

*For any subsets $U \subseteq \mathcal{V}$ and $X \subseteq \mathcal{X}$, let us define $(U:\pi) := \{\, (u:a) \mid u \in U,\ a \in \pi \,\}$ and $\mathcal{X}(U) = \{\, x \mid t.x \in U \,\}$. Let us also denote $\overline{X} := \mathcal{X} \setminus X$, and $\mathbb{Z}.X := \{\, t.x \mid t \in \mathbb{Z}, x \in X \,\}$. Given any $u = t.x \in \mathcal{V}$, let us denote $t'.u$ for $(t' + t).x \in \mathcal{V}$.*




(a) $G$ (b) $A_{x_1} G$

Fig. 3: A local rule application $A_{x_1}$ is a function from the entire graph to the entire graph which only affects the vertices at position $\mathcal{N}_{x_1}(G)$. These are either interior vertices (circled in dark blue) or boundary vertices (circled in cyan). These boundary vertices, and their external edges, must be preserved, although their internal (here white/black) internal states can be update. The whole interior can be modified almost arbitrarily.

**Definition 2 (Graphs, Past).** *A* graph *$G$ is given by a tuple $(\mathrm{I}_G, \mathrm{B}_G, \mathrm{E}_G, \sigma_G)$ where:*

- $\mathrm{I}_G \subseteq \mathcal{V}$ *has its elements called* internal vertices *of $G$,*
- $\mathrm{B}_G \subseteq \mathcal{V}$ *has its elements called* border vertices *of $G$,*
- $\mathrm{E}_G \subseteq ((\mathrm{I}_G \cup \mathrm{B}_G):\pi)^2 \setminus (\mathrm{B}_G:\pi)^2$ *has its elements called* (oriented) edges*, and*
- $\sigma_G : \mathrm{I}_G \to \Sigma$ *maps each internal vertex to its states.*

*We denote $\mathrm{V}_G := \mathrm{I}_G \cup \mathrm{B}_G$. This tuple has to be such that:*

- *Vertex partitioning:* $\mathrm{I}_G \cap \mathrm{B}_G = \emptyset$*,*
- *Unicity of positions:* $\forall t.x, t'.x' \in \mathrm{V}_G,\ x = x' \Rightarrow t = t'$*,*
- *Port non-saturation:* $\forall (u:a, v:b), (u':a', v':b') \in \mathrm{E}_G,\ u:a \neq v':b' \ \wedge\ u:a = u':a' \Leftrightarrow v:b = v':b'$*,*
- *Border attachment:* $\forall u \in \mathrm{B}_G,\ \exists (v:a, v':a') \in \mathrm{E}_G,\ u \in \{\, v, v' \,\}$*, and*
- *Acyclicity:* $\forall n \in \mathbb{N},\ \forall \langle (u_i : a_i, v_i : b_i) \rangle_{i \in \{1,\dots,n+1\}} \in {\mathrm{E}_G}^{n+1}$ *s.t.* $(\forall i \in \{1,\dots,n\},\ v_i = u_{i+1}),\ u_1 \neq v_n$*.*

*We denote by $\mathcal{G}$ the set of all graphs, and by* $\mathrm{Past}(G) \subseteq \mathrm{V}_G$ *the vertices of $G$ with no incoming edges.*

Summarizing, each position $x$ only appears once, each vertex port $:a$ can only be used once per vertex, and border vertices are only there to express dangling edges of internal vertices. The $\mathrm{Past}(G)$ vertices intuitively stand for computational processes that are no longer awaiting for results by others, and are therefore ready to be executed.

**Notations.** Given a subset $U \subseteq \mathcal{V}$, we define the induced subgraph $G_U \sqsubseteq G$ as the graph whose internal vertices $I_{G_U}$ are given by $I_G \cap U$, and whose edges $E_{G_U}$ are all those edges of $G$ which touch a vertex in $I_{G_U}$. Thus, its border vertices $B_{G_U}$ are those nodes of $V_G \setminus I_{G_U}$ which lie at distance one of $I_{G_U}$ in $G$. For a subset $X \subseteq \mathcal{X}$, we write $G_X$ for the induced subgraph $G_{\mathbb{Z}.X}$ (see Fig. 2).
We also introduce the operation $G \sqcup H$, which is only defined if $G$ and $H$ can both be obtained as induced subgraphs of the some graph, in which case $G \sqcup H$ is the smallest such graph. In particular, this implies that if $u \in I_G$, $v \in \mathrm{B}_G$, $v \in I_H$ and $(u\!:\!a, v\!:\!b) \in E_G$, it must be the case that $(u\!:\!a, v\!:\!b) \in E_H$ and $u \in V_H$. This union will allow us to express locality.
We denote $X^-$ the set $\{x \in X \cap \mathcal{X}(\mathrm{I}_G) \mid \mathrm{V}_{G_x} \subseteq \mathbb{Z}.X\}$, and call these the interior vertices of $X$ in $G$. Its other vertices, namely $X \setminus X^-$, are referred to as the boundary of $X$ in $G$, see Fig. 2. Notice that the notion of border and the notion of boundary are quite different.

Notice that $G = G_X \sqcup G_{\overline{X}}$. This decomposition will allow us to define the action of our local rules. We will proceed as follow. First we define a *neighbourhood scheme* $\mathcal{N}$ which, given a position $x$, selects a subset of nearby positions $\mathcal{N}_x(G)$. Second we define *operators* $A_{(-)}$ which, given a position $x$, rewrite the graph $G$ as $A_x G$. Third we say that $A_{(-)}$ is $\mathcal{N}$*-local* if the action of $A_x$ is to replace just the left hand side of $G = G_{\mathcal{N}_x(G)} \sqcup G_{\overline{\mathcal{N}_x(G)}}$, independently of the right hand side—an operation which can likely be formalised as a double push-out [8].

**Definition 3 (Neighbourhood scheme).** *A* neighbourhood scheme $\mathcal{N}$ *is an operator from* $\mathcal{X} \times \mathcal{G}$ *to* $\mathcal{P}(\mathcal{X})$ *mapping a pair* $(x, G)$ *to* $\mathcal{N}_x(G) \subseteq \mathcal{X}$ *such that :*

- *Reachability: For every* $y \in \mathcal{N}_x(G)$ *there exists a directed path from* $x$ *to* $y$*.*
- *Extensivity:* $G_{\mathcal{N}_x} \sqsubseteq H \sqsubseteq G$ *implies* $G_{\mathcal{N}_x} = H_{\mathcal{N}_x}$;

*When* $G$ *is clear from context, we write* $\mathcal{N}_x$ *instead of* $\mathcal{N}_x(G)$*.* $\mathcal{N}_x^-$ *refers to the internal vertices of* $\mathcal{N}_x$ *with respect to* $G$*.*

Notice that our definition of neighbourhood schemes is quite permissive. It even allows for unbounded, or infinite neighbourhood (if the directed edges are interpreted as lightlike this still respects bounded speed of propagation of information), even though for all practical purposes one is likely to use a bounded neighbourhood. Our results will apply so long as the abstract conditions are met. For some of them we need extensivity, which is there to forbid that global criteria be used to decide whether some closeby vertex belongs to the neighbourhood or not. It demands that the neighbourhood $\mathcal{N}_x(G)$ as computed by the function $\mathcal{N}$ over a graph $G$, be the same as that computed over any subgraph $H$ of $G$ that comprises the neighbourhood. It can be understood as a form of graph-locality of the neighbourhood scheme $\mathcal{N}$ itself. Typically, if $\mathcal{N}_x(G)$ is computed step by step starting from $x$ until 'hitting wall' i.e. a local ending criterion, then it will be strongly extensive, i.e. $G_{\mathcal{N}_x} \sqsubseteq G$ implies $G_{\mathcal{N}_x} = H_{\mathcal{N}_x}$. Extensivity is a bit more permissive, e.g. in a segment, we can grab the first dozen closest neighbours if they exist, and decide to keep none of them otherwise.

Next we define the notion of a local rule $A_{(-)}$. Before we proceed, remember that our graphs can never contain both vertices $u = t.x$ and $u' = (t+1).x$. It follows that the local rule $A_x$ will act unambiguously on the unique vertex of the form $u = t.x$. Keep in mind also that our directed edges stand for dependencies between events, i.e. if $u = t.x$ points

to $v = t'.y$, then $v$ is considered ahead in time of $u$, and thus frozen awaiting information from $u$. The action of some local rule $A_y$ on $v$ is therefore trivial, preventing $y$ to be computed too far ahead and thereby providing a weak synchronisation mechanism. The action of $A_x$ on $u$ is non-trivial only if $u$ is minimal aka Past. Such a $u$ can be thought of as lagging behind in time, and no longer awaiting for any information—it has reached its local normal form. The action of $A_x$ is to dispose of it by communicating its information to $v$ and other dependencies, and creating vertex $u' = (t + \Delta t).x$ in some provisional state.

**Definition 4 (Local rule).** *A* local rule *is an operator over graphs $A_{(-)} : \mathcal{X} \to (\mathcal{G} \to \mathcal{G})$ which is $\mathcal{N}$-local for some neighborhood scheme $\mathcal{N}$, i.e. for all $G$ in $\mathcal{G}$, for all $x$ in $X$,*

$$A_x G = \begin{cases} (A_x G_{\mathcal{N}_x}) \sqcup G_{\overline{\mathcal{N}_x}} & \text{if } x \in \mathrm{Past}(G) \\ G & \text{otherwise} \end{cases}$$

*where the action of $A_x$ on $G_{\mathcal{N}_x}$ must be* context-preserving—i.e. *it preserves the border $B_G = B_{A_x G}$ and its border edges $e \in \mathrm{E}_G \setminus (\mathrm{I}_G : \pi)^2 \implies e \in \mathrm{E}_{A_x G}$. From now on $A_{yx}$ will stand for $A_y A_x$. We say that $\omega \in \mathcal{X}^*$ is a valid sequence in $G$ if, for all $\omega_1, \omega_2 \in \mathcal{X}^*$ such that $\omega = \omega_2 x \omega_1$, we have $x \in \mathcal{X}(\mathrm{Past}(A_{\omega_1} G))$. We denote $\Omega_G(A) \subseteq \mathcal{X}^*$ the set of valid sequences in $G$.*

The fact that $A_x(G_{\mathcal{N}_x}) \sqcup G_{\overline{\mathcal{N}_x}}$ needs to exist for any $G$ implies that $A_x(G_{\mathcal{N}_x})$ and $G_{\overline{\mathcal{N}_x}}$ must "agree". A consequence is that new vertices $V_{A_x G} \setminus V_G$ are exclusively vertices at positions $\mathcal{N}_x^-(G)$ with time tags modified (e.g. the vertices circled in dark blue in Fig. 3). More precisely,

**Lemma 1. (Position preservation).** *Let $A_x : \mathcal{G} \to \mathcal{G}$ be a local rule, then $\mathcal{X}(I_{A_x G}) \subseteq \mathcal{X}(I_G)$ and all new names are at positions of ${\mathcal{N}_x}^-$—i.e. $\mathcal{X}(I_{A_x} \setminus I_G) \subseteq {\mathcal{N}_x}^-$.*

An additional property that can be required, without difficulty, of our operators, is renaming-invariance [4]. We will skip this here.

Having defined the considered graphs and the kind of local transformations allowed on them, let us state our goal informally. Consider a graph $G$, a local rule $A_{(-)}$ and all possible sequences $\omega, \omega', \ldots \in \Omega_G(A)$. If one applies $A_\omega$, one obtains one possible evolution of the system. But $A_{\omega'}$, $A_{\omega''}$,... are equally legitimate different orders of local rule applications. We aim to define precisely what it means for all these possible evolutions to actually agree on a consistent common story, i.e. a *consistent space-time diagram*. To motivate the remaining formalization, we consider examples. Later we give sufficient conditions for a local rule to induce such consistent space-time diagrams.

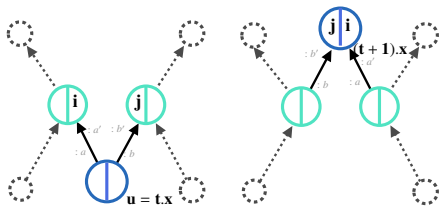
**Definition 5 (Space-time diagram).** *Given a graph $G$ and a local rule $A_{(-)}$,* space-time diagram *$\mathcal{M}_A(G) := \{ A_\omega G \mid \omega \in \Omega_G(A) \}$ is the set of all generated graphs. We sometimes omit $A$ and $G$.*

To visualize how the graphs in $\mathcal{M}$ share common vertices and edges, some *space-time backgrounds* are depicted (Figs 5, and 9). These space-time backgrounds are "pseudo-graphs", i.e. graph in a common sense, without any of the port-constraints of Def. 2 imposed. Each pseudo-graph $M$ is defined by:

$$V_M = \bigcup_{G \in \mathcal{M}} V_G \qquad\qquad E_M = \bigcup_{G \in \mathcal{M}} \mathrm{E}_G.$$

## 3 Particle system example

We now show how asynchronous applications of a local rule can represent a dynamical system of left and right-moving particles, consistently, thereby fixing issues of Fig. 1.




(a) $G_{\mathcal{N}_{x_1}}$. (b) $A_{x_1} G_{\mathcal{N}_{x_1}}$.

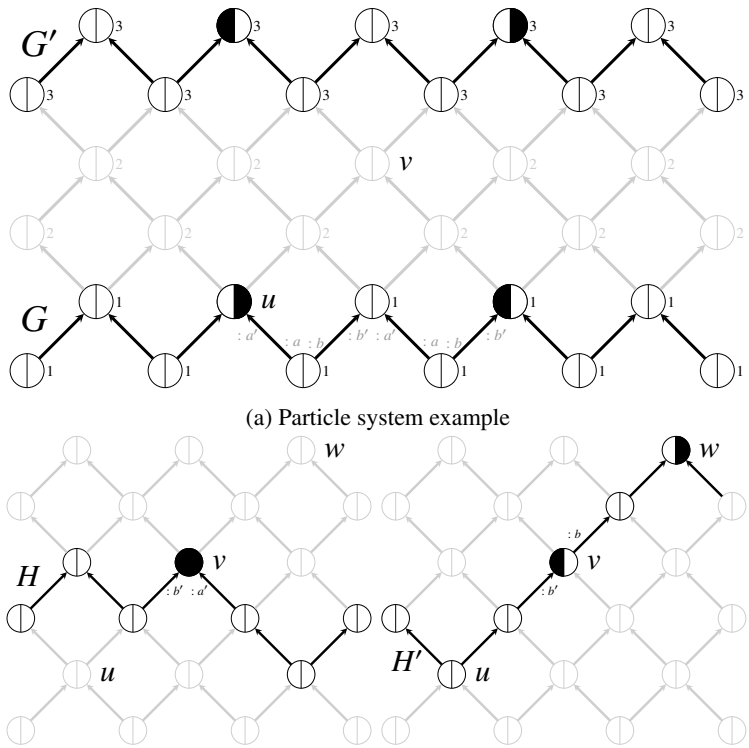
Fig. 4: *The local rule for the particle system.* $A_x$ acts by consuming the internal state $i = \sigma^r_G(v)$ of vertex $v$ (and symmetrically with $j = \sigma^l_G(w)$), thereby moving those particles at $x$ if they are present. It also updates ports from $a, b$ to $a', b'$, flips the arrows pointing to $x$, and increments its timetag, in order to move the vertex from past $u = t.x$ to future $u' = 1.u = (t + 1).x$. Dashed edges and vertices do not influence the local rule.

Again the vertices of our graphs have names of the form $u = t.x$, they must be thought of as events in space-time. Here internal states are pairs of bits representing the presence of a left-moving particle or not, and of a right-moving particle or not, i.e. $\sigma_G(u) = (\sigma^l_G(u), \sigma^r_G(u))$. Each edge goes either from port $:a$ to $:a'$ or from port $:b$ to $:b'$, thereby indicating a spatial direction ($a$ versus $b$) and a temporal orientation (unprimed versus primed). Altogether each graph must be thought of, not as a space-time diagram, but as a 'space-like cuts' of a space-time diagram, i.e. a snapshot, cf. Fig. 5a.

The local rule is given by Fig. 4. It allows us to evolve one graph $G$, understood as a space-like cut (a snapshot), into another later space-like cut $G'$, as in Fig. 5a. It is possible to convince oneself that any sequence of applications of the local rule in the vicinity of the two particles leads the collision occurring at $v$ and nowhere else.
Notice also that the graph $G$ can be evolved asynchronously into either $H$ as in Fig. 5b or $H'$ as in Fig. 5c. These graphs agree upon the trajectory of the moving particles. Let us now turn our attention towards considerations that we eventually formalize in Sec. 6. By the above two remarks, this local rule feels "space-time deterministic", as there is a sense in which it produces well-determined events in space-time. But this property is a bit subtle to formulate, e.g. the state associated to the event $v$ seems different in $H$ and $H'$, as the particle got 'consumed' from $v$ to $w$. Still, the state of $v$ remains a function of the way it gets traversed by the snapshot $H$ or $H'$, as represented here by its set of incoming ports to $v$. This motivates the notions of full consistency and full space-determinism which we give in Def. 6.

Instead of entering this subtlety, we could have restricted ourselves to local rules that do not consume the particle, e.g. leaving a trail of the left-mover at $v$ in $H'$. But the 'particle consumption mechanism' of this example is something that we do want to be able to capture. This, we admit, is for reasons that lie beyond the scope of this paper, but we can mention them in passing: 1/ In quantum theory, if the particle is not consumed as it moves, it leaves a trail behind that entangles several sites. Due to this

entanglement, ignoring the trail decoheres the particle, making it to behave classically. This is essentially the no-cloning theorem. We want to make sure this work is compatible with a quantum extension. 2/ In quantum field theory, it is also the case that the quantum state associated to a point in space-time is well-determined only if we specify the angle at which it gets cut by the spacelike cut. The study of this example sets the aim of the paper: to understand when asynchronism meets space-time determinism.




(a) Particle system example

(b) $\sigma_H(v) = (0, 1) \ldots$ (c) but $\sigma_{H'}(v) = (0, 0)$.

Fig. 5: *Particle system example (a):* In black we highlight just the graphs $G$ and $G'$ belonging to the space-time diagram $\mathcal{M}_A(G)$. The local rule here moves the left-side particle towards the right and the right-side particle towards the left. Note how the problem raised by Fig. 1 is solved. The only point in space-time which can contain both particles is $u$. *States depend on the cut (b)(c):* Both graphs $H$ and $H'$ belong to the same space-time diagram $\mathcal{M}_A(H_0)$ with $H_0$ containing a right-moving particle in $u$. They both contain vertex $v$. In $H'$, the particle that was present in $H$ has moved to point $w$. The state associated to $v$ thus depends on the way it is cut, which is captured by its set of incoming ports e.g. $\{a', b'\}$ versus just $\{a'\}$.

## 4 Simulating synchronous cellular automata

Beyond the above particle system example, we now show that we can simulate any 1D cellular automata (CA for short) in spite of its synchronicity. The construction borrows to the well-studied 'marching soldiers' scheme [18], as best formalised by [29] , but relies on the DAG of dependency rather than extra states as its mechanism for local, relative synchronisation. Without loss of generality [21], we simulate radius half CA.

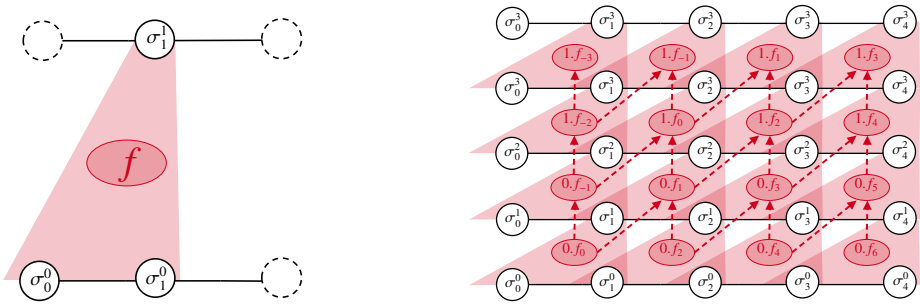
First let us recall that a radius one half cellular automaton acting on an alphabet $\Sigma_{CA}$ is a global function $F : {\Sigma_{CA}}^{\mathbb{Z}} \to {\Sigma_{CA}}^{\mathbb{Z}}$ defined by the synchronous application of a local function $f : {\Sigma_{CA}}^2 \to \Sigma_{CA}$ everywhere at once, cf. Fig. 6a. The dependencies between the different applications of $f$ are shown in Fig. 6b, notice the similarity with Fig. 5a. This suggests that the natural way to encode this cellular automaton in our model is to use vertices (a.k.a events) to represent each application of $f$.

Second we define $\Sigma = (\Sigma_{CA} \cup \epsilon)^2$ and pick the same neighbourhood scheme and ports as in the previous example. We define the local rule as depicted in Fig. 7a and Fig. 7b. Its action can be understood as follow: 1/ applied to the vertex named $0.f_0$, it applies $f(\sigma_0^0, \sigma_1^0)$ 2/ and stores the result both in the right part of the vertex $0.f_{-1}$ and in the left part of the vertex $0.f_1$. 3/ Finally it creates the vertex $1.f_0$, which is awaiting for the output of the $0.f_{-1}$ and $0.f_1$ applications, as represented by the two incoming edges.

We encode the initial configuration $\sigma^0$ by a graph $G$ (see Fig. 7c) which contains two copies of each internal state in $\sigma^0$. The dynamics will indeed generate every configuration $\sigma^i = F^i(\sigma^0)$. The space-time diagram is similar to that of Fig. 5a. The scheme can easily be generalised to $d$-dimensional CA, and is likely to work for any local synchronous evolution in a broad sense ; the idea being to use vertices to encode applications of the local rule, and edges to represent applications causal relationships.

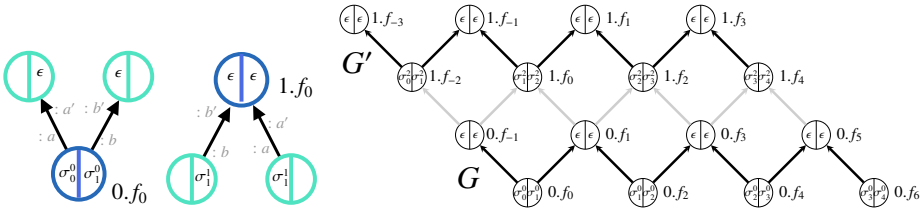
## 5 Beyond synchronous simulation

The previous examples are simulations of synchronous systems. Beyond these, we are interested in expressing systems that remain deterministic but are genuinely asynchronous. Such local rules still have fully consistent space-time diagrams: the only thing which is lost is the possibility to interpret these through the lens of a global clock. A paradigmatic, physics-inspired example for this whole class of systems is the following time dilation example. The time dilation example extends the internal state space $\Sigma$ used in Sec. 3 with two more states: the green and red states. The same local rule is extended so that if one such green/red particle is found at position $x$, it will stay there, oscillating between both colours and altering the very texture of space-time, cf. Fig. 8. This results in time dilation as can be seen from Fig. 9. For instance, if two identically made clocks were modeled out of a signal oscillating from left to right and right to left between two neighbouring nodes, the clock lying on the right-hand-side of the green/red particle would tick twice as slowly as the one lying on the left-hand-side. Yet, the very same local rule is being applied left and right of the green/red particle. Such a phenomenon is reminiscent of general relativity, e.g. time flows slightly slower on Earth than it does in the stratosphere, as measured by identically made atomic clocks. Yet, the same laws of Physics apply in the stratosphere and on Earth. How did we get there?




(a) Radius one half locality . . . (b) . . . results in the above global evolution.

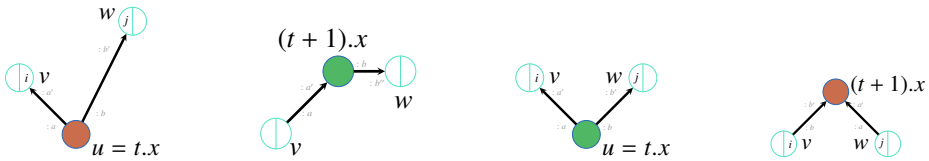
Fig. 6: *Radius one half local rule (a):* A radius one half local rule $f$ takes two states as input ($\sigma_0^0, \sigma_1^0 \in \Sigma_{CA}$) and outputs $\sigma_1^1 = f(\sigma_0^0, \sigma_1^0)$. *Global evolution (b):* The initial configuration is an infinite one dimensional array $(\sigma_i^0)_{i\in\mathbb{Z}}$. We consider the cellular automaton which apply $f$ homogeneously in space. In black we depicted the successive configurations computed by this automaton ($\sigma_k^i$ is just short for $f(\sigma_{k-1}^{i-1}, \sigma_k^{i-1})$). In red we drew each time the local rule $f$ gets applied. We gave a name to each occurrence to introduce more easily the simulation scheme of Fig. 7c. Finally we used red arrows to depict the dependencies between each occurrence of $f$, for example since $0.f_1$ needs the output $\sigma_1^1$ of $0.f_0$ to compute, we drew an arrow $0.f_0 \to 0.f_1$.




(a) $G_{\mathcal{N}_{f_0}}$. (b) $A_{f_0} G_{\mathcal{N}_{f_0}}$. (c) Initial graph and space-time diagram.

Fig. 7: *The local rule for CA simulation (a) and (b):* $A_{f_0}$ acts exactly as $A_x$ in Fig. 4, except for the internal states. If $\sigma(0.f_0) = (\sigma_0^0, \sigma_1^0)$ does not contain $\epsilon$, it computes $\sigma_1^1 = f(\sigma_0^0, \sigma_1^0)$ and stores the result in both neighbours. *Space-time diagram of the simulation (c)* of the cellular automaton in Fig. 6b. The highlighted $G$ encodes the initial cellular automaton configuration $\sigma^0$ of the CA, whilst $G'$ encodes $\sigma^2$ the configuration obtained after two time steps of the cellular automaton.

We argue that, to some extent, the construction of this model of computation mimics some of the key steps of the derivation of general relativity theory from physical symmetries—as found in standard textbooks [28]. The developed analogy can safely be skipped by the reader with lesser interest in Physics. Indeed, let us remind the reader that the textbook derivation proceeds by: 1/ Assuming the existence of a well-determined space-time. 2/ Requiring covariance, i.e. invariance under changes of coordinates, which implies a form of asynchronism as one can choose coordinates whereby one region




(a) $G_{\mathcal{N}_x}$. (b) $A_x(G_{\mathcal{N}_x})$. (c) $H_{\mathcal{N}_x}$. (d) $A_x(H_{\mathcal{N}_x})$.

Fig. 8: *The local rule for time dilation.* We define here the behaviour of $A_x$ when $u = t.x$ is colored—i.e. $\sigma(u) = green \vee \sigma(u) = red$. In both the cases $(a)$ and $(c)$, incoming particles get destroyed and we flip the color at $x$. On a green vertex case $(c)$, $A_x$ behaves as expected. On a red vertex case $(a)$ $A_x$ creates an anomaly. The edge between $(t + 1).x$ and $w$ is reversed, thus we will be forced to update $v$ once more, before we update $w$.

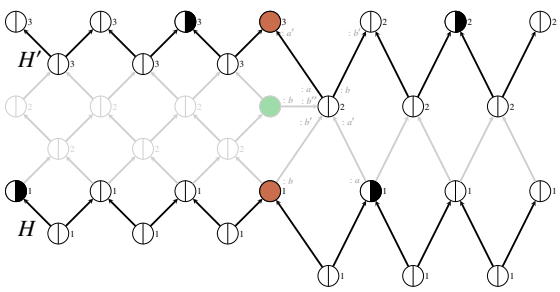



Fig. 9: *Time dilation example.* In black we highlight two graphs $H$ and $H'$ belonging to the space-time diagram. We start with two particles, one left and one right of the red. Notice how, even if the same local rule gets applied everywhere, times flows twice as fast for the left particle.

of a space-like cut will evolve (large time lapse), but not the other (small time lapse). 3/ Concluding that in order to obtain covariance, one needs to provide extra causality structure at each point, namely the metric field. 4/ Assuming background-invariance, namely enabling the possibility that space-time be curved by the presence of this newly allowed metric. 5/ Providing a dynamic upon the metric itself.
Here, in the discrete, we: 1/ enforced a well-determined space-time, 2/ in spite of an asynchronous evaluation strategy, 3/ by introducing an extra causality structure, namely a DAG of dependencies. 4/ We then allowed ourselves to consider graphs with exotic such DAG, and 5/ rules manipulating them.

## 6 Obtaining space-time determinism

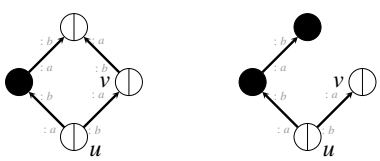
The space-time determinism of a local rule $A_{(-)}$ is the idea that the graphs that it generates from a seed $G$, which altogether form a space-time diagram $\mathcal{M}_A(G)$, are consistent with one another, as regards the events that they both describe. One might have hoped for a naive notion of consistency of $\mathcal{M}_A(G)$, whereby any two graphs of the space-time diagram must agree on the state of a vertex $u \in \mathcal{V}$, if it so happens to appear in both of them. But the example in Sec. 3 (Fig. 5) shows that things are more subtle. Its discussion motivates a definition of full consistency whereby any two graphs of $\mathcal{M}_A(G)$ must agree on the state of a vertex $v$ (in terms of its neighbourhood and internal state) whenever they agree on the set of incoming ports to that vertex $v$. In particular this implies that the state

of $v$ should be the same for every graph of $\mathcal{M}_A(G)$ such that $v \in \mathrm{Past}(G)$, which can be understood as stating that the "result state" (a.k.a. normal form) at $v$ is well-determined. We refer to this weaker demand as weak consistency (see Fig. 10).

**Definition 6 (Consistency).** *Two graphs $G$ and $H$ are* fully consistent *iff for all $v \in \mathrm{I}_H \cap \mathrm{I}_G$:*

$$\pi.\mathrm{E}_G(v) = \pi.\mathrm{E}_H(v) \Longrightarrow G_v = H_v,$$

*where $\mathrm{E}_G(v)$ is the set of edges ending in $v$, and $\pi.\mathrm{E}_G(v)$ denotes the set of incoming ports of $v$, i.e. $\pi.E := \{\, b \mid (u : a, v : b) \in E \,\}$. Full consistency is denoted $G \mathbin{/\!/} H$. The graphs $G$ and $H$ are called weakly consistent if they respect this condition in the special case where $\mathrm{E}_G(v) = \emptyset$. We say that a space-time diagram $\mathcal{M}$ is fully (respectively* weakly*)* consistent *if each pair of graphs in $\mathcal{M}$ is fully (resp. weakly) consistent. Finally a local rule $A_{(-)}$ is said to be fully* (resp. weakly) space-time deterministic *if and only if for all graphs $G$, $\mathcal{M}_A(G)$ is fully (resp. weakly) consistent.*




(a) $G$ (b) $H$

Fig. 10: *Weak consistency versus full consistency.* The set of graphs $\{G, H\}$ is weakly consistent but not fully consistent. Indeed we have $G_u = H_u$ but full consistency fails in $v$ because we have $\mathrm{E}_G(v) = \mathrm{E}_H(v) = \{ : a \}$ but $G_v \neq H_v$.

We now embark in the quest for a set of properties ensuring that an $\mathcal{N}$-local rule $A_{(-)}$ generates only fully consistent space-time diagrams. We start with two properties. The first one asks for timetags to only increase. The second one, akin to strong confluence or sequential independence in parallel graph transformations, states that independent rule applications on a graph $G$ applied in any order lead to the same graph.

**Definition 7 (Time-increasing commutative local rules).** *A local rule $A_{(-)}$ is*

- time-increasing *iff $\forall t.y \in V_G$, $\forall t'.y \in V_{A_x G}$, $t \leq t'$, with $t < t'$ if $y = x$;*
- commutative *iff $\forall x, y \in \mathcal{X}(\mathrm{Past}(G))$, $A_x A_y(G) = A_y A_x(G)$.*

Notice that a past vertex must remain so by locality, hence the well-definiteness of commutativity. These two properties place strong constraints on past vertices of a space-time diagram $\mathcal{M}_A(G)$: they entail weak consistency. Moreover each space-like cut is determined by its set of past elements.

**Proposition 1. (Obtaining weak consistency).** *Let $A_{(-)}$ be a time-increasing commutative local rule. For all graphs $G$, $\mathcal{M}_A(G)$ is weakly consistent.*

**Proposition 2. (Pasts determine space-like cuts)** *Let $A_{(-)}$ be a time-increasing commutative local rule, $G$ a graph and $H, J \in \mathcal{M}_A(G)$. If $\mathrm{Past}(J) = \mathrm{Past}(H)$ then $J = H$.*

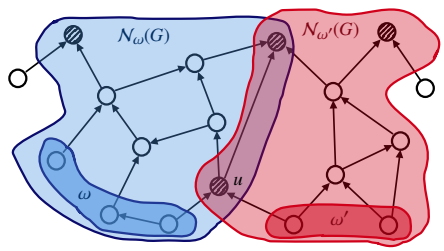



Fig. 11: *Privacy and port decreasing.* (*a*) Given a set of disjoint vertices $\omega'$ privacy demands that the neighbourhood $\mathcal{N}_\omega$ (blue) and $\mathcal{N}_{\omega'}$ (red) only intersect on their boundaries (hatched vertices). (*b*) The port decreasing condition demands that, in order to modify a vertex, $A_\omega$ must pay the price of decreasing one of its incoming private ports. Here, $u \in G_{\mathcal{N}_\omega} \cap G_{\mathcal{N}_{\omega'}}$ can be modified by both $A_\omega$ and $A_{\omega'}$ as each of them has private access to it.

However, in the example of Sec. 3, weak consistency is trivially realized even for non-trivial dynamics. This example motivated the definition of full consistency. In order to have local conditions that entail full consistency, we ask that neighbourhood schemes be private and that local rules be port decreasing. Note that these properties hold for the neighbourhood schemes and local rules used in Secs 3, 4 and 5.

In the following we denote $\mathcal{N}_\omega(G) = \bigcup_{\beta x \alpha = \omega} \mathcal{N}_x(A_\alpha G)$ the union of the neighbourhood of each vertex $x \in \omega$, at the time $A_x$ is to be computed. Privacy then demands that any disjoint sequences $\omega, \omega'$ have their $\mathcal{N}_\omega(G)$ and $\mathcal{N}_{\omega'}(G)$ intersecting only on the vertices of their boundaries (see Fig. 11), a property akin to parallel independence [15]. Privacy ensures that concurrent influences happen only at these joint boundaries, an essential ingredient of full consistency, as illustrated in Fig. 12a and 12b. They also make it easier to check whether two valid operator $A_x$ and $A_y$ commute. Indeed when $\mathcal{N}_y(G) = \mathcal{N}_y(A_x G)$ and $\mathcal{N}_x(G) = \mathcal{N}_x(A_y G)$, privacy allows us to only check commutation on the boundaries to ensure it globally.

**Definition 8 (Privacy).** *A neighbourhood scheme $\mathcal{N}$ is private, iff for any graph $G$ and any disjoint valid sequences $\omega, \omega' \in \Omega_G(A)$, we have $\mathcal{N}^-_{\omega'}(G) \cap \mathcal{N}_\omega(G) = \emptyset$.*

Privacy may seem hard to check at first, especially given that $\mathcal{N}_\omega(G)$ depends on $A_{(-)}$. In the long version of this paper we introduce a concrete $\mathcal{N}'_\omega(G)$ which does not depend on $A_{(-)}$ as it assumes the "worse-case scenario" for it. Since $\mathcal{N}'_\omega(G)$ is private, it suffices to check that $\mathcal{N}_x(G) \subseteq \mathcal{N}'_x(G)$ to prove its privacy.

Lastly we want to forbid that a local rule modifies a vertex without altering its incoming ports, as this would immediately infringe full consistency (see Fig. 12c and 12d). Moreover the modification needs be decreasing—otherwise we could apply $A_x$ twice in a row and get to the exact same counterexample. This idea that $A_x$ should decrease the incoming ports of the vertex $v$ it modifies can be understood as a natural way of 'locally reflecting the progress of the computation of $v$', i.e. geometrically accounting for the fact that the dependency between $x$ and $v$ has been reduced, and hence their space-time relationship has changed. In Sec. 3 the local rule is port decreasing because we suppress an incoming edge each time we modify a vertex. In Sec. 5 the local rule is port decreasing for more subtle reasons in the case of a red vertex: we decrease $b'$ into $b''$ (see Fig. 8a and 8b).

(a) $G$ (b) $A_uG/A_vG$ (c) $G$ (d) $A_uG$

Fig. 12: *Inconsistent dynamics examples.* The example of $(a)\&(b)$ relies on a neighbourhood scheme which is not private as $w \in \mathcal{N}_u^-(G) \cap \mathcal{N}_v^-(G)$. It follows that $A_uG$ and $A_vG$ disagree on the internal states of $w$, whilst both updating its incoming port in the same fashion ($b$ becomes $a < b$). The example of $(c)\&(d)$ is not port decreasing. It follows that $G$ and $A_uG$ disagree on $v$ but its set of incoming ports stay the same.

**Definition 9 (Port decreasing).** *An $\mathcal{N}$-local rule $A_{(-)}$ is* port decreasing *iff for any $G \in \mathcal{G}$, $x \in \mathcal{X}(\mathrm{Past}(G))$ and $u \in V_G \cap V_{A_xG}$ such that $G_u \neq (A_xG)_u$, we have*

$$\pi.(\mathrm{E}_G(V_G, u) \setminus \mathrm{E}_G(\overline{\mathcal{N}_\omega^-}, u)) > \pi.(\mathrm{E}_{A_xG}(V_{A_xG}, u) \setminus \mathrm{E}_G(\overline{\mathcal{N}_\omega^-}, u))$$

*for an order $\leq$ over sets of ports which is for any $A, B, A', B' \in \mathcal{P}(\pi)$:*

- ***inclusive*** *i.e. $A \supseteq A' \implies A \geq A'$*
- ***monotonous*** *i.e. $A \cap B = \emptyset \ \wedge \ A \geq A' \ \wedge \ B > B' \implies A \cup B > A' \uplus B'$*

*where $\mathrm{E}_G(Y, u)$ is the set of edges in $G$ from any $v \in \mathbb{Z}.Y$ to $u$, and $\pi.E$ is as in Def. 6.*

Note we ask a port decreasing operator to decrease the port of a *private edge*, i.e. an edge starting from $\mathcal{N}^-$. Without this assumption we can give a counterexample by considering the graph of Fig. 11. Say both $A_\omega$ and $A_{\omega'}$ were to modify the internal state of $v$ and decrease the port associated to the shared edge coming from $u$. Then we could have $\pi.\mathrm{E}_{A_\omega G}(v) = \pi.\mathrm{E}_{A_{\omega'}G}(v)$ whilst $\sigma_{A_{\omega'}G}(v) \neq \sigma_{A_\omega G}(v)$. Finally we state our main result.

**Theorem 1. (Obtaining full consistency)** *Let $A_{(-)}$ be a port-decreasing time-increasing commutative $\mathcal{N}$-local rule, with $\mathcal{N}$ a private neighbourhood scheme. For all graphs $G$, $\mathcal{M}_A(G)$ is fully consistent.*

The proof is quite intricated, but we can give the following intuition. On the one hand the commutation hypothesis ensures that $A_\omega G \parallel A_{\omega'}G$ as long as $\omega'$ is a permutation of $\omega$. On the other hand as long as $\mathcal{N}$ is extensive, monotonous and private two disjoint operators necessarily modify different subgraphs, albeit with a common boundary. Then, by decreasing the private incoming port of each modified vertex, we obtain full consistency by dismissing its premise.

Hence, strong confluence augmented with a number of hypotheses leads to full consistency. Indeed, full consistency is a stronger requirement than confluence, as it shifts the focus from global configurations comparison to the structured, local comparison of spacetime events.

## 7 Conclusion

*Summary of results.* We introduced graphs that can be thought of as space-like cuts of space-time diagrams. The vertices have names of the form $u = t.x$, they can be

thought of as events. The edges can be thought of as dependencies between events, i.e. if $u = t.x$ points to $v = t'.y$, then $v$ is ahead in time of $u$, awaiting for information from $u$. The action of a local rule $A_x$ on $u$ is non-trivial only if $u$ is minimal: it disposes of it by communicating its information to $v$ and other dependencies, and creates vertex $u' = (t + 1).x$ in some provisional state.

We argued that the right notion of full space-time determinism is the full consistency of its space-time diagram: the state of each event (in terms of its internal state and connectivity) needs be a function of its set of incoming ports, as these represent the angle at which the space-like cut traverses the event. We gave sufficient conditions for the asynchronous applications of a local rule to be fully space-deterministic: they must be commuting and port-decreasing, with respect to an extensive, monotonous, private notion of neighbourhood. We also looked at a weaker notion of consistency, requiring that only the normal forms of events across space-time be well-determined: commutation alone suffices then.

Throughout, we argued of the potential implications for distributed computation (weak space-time determinism), asynchronous simulation of dynamical systems (full consistency and our first and second example), and discrete toy models of general relativity (our third example).

*Related works.* Geometry is dynamical in our work. We thus hope it makes useful addition to the already wide literature on Graph Rewriting [34,14]. We are aware that the dominating vocabulary to describe them is now that of Category theory, in which notions of space-time diagrams have been developed e.g. [7].

We instead use the vocabulary of dynamical systems, as we came to consider Graph Rewriting through a series of works generalising CA to synchronous, causal graph dynamics [2], and tilings to graph subshifts [5]. We are confident that abstracting away the essential features of our formalism could yield interesting categorical frameworks, e.g. à la [26].

The closest works however turn out to come from varied communities. In algorithmic complexity [30] uses a DAG of dependencies representation to reduce the synchronisation costs of simulating a class of synchronous algorithms—we use it in the more context of dynamical systems in order to relax synchronism altogether, whilst safeguarding space-time determinism. In computational Physics [19] promotes the lattice of dependencies of local rule applications to a notion space-time, and advocates a notion of 'causal invariance' based on the unicity of this lattice, as formalised in the context of string rewriting [33]—we formalise space-determinism for graph rewriting mathematically, without reference to this lattice of dependencies of applications, and provide local conditions to achieve it. In the network reliability community, [18] obtains a result similar to Prop. 1—our local rules are allowed to modify the neighbouring vertices and the graph per se, and we reach full consistency.

*Perspectives.* Clock synchronisation is an expensive overhead for parallel simulation of dynamical systems, as well as numerous distributed computation applications. The hereby developed theoretical framework says we can just do away with them and still obtain strong forms of space-time determinism, provided that the local rule meets certain requirements. We are looking forward to see this being applied in practice. Along the way of practical applications, one ought to first try and adapt existing graph rewriting

software tools [32,16,11] for the sake of automatically checking full consistency.
We, on the other hand, are likely to focus on the reversible and quantum regimes of these graph rewriting models. Another, important open problem is to understand whether space-times of fully space-time deterministic local rules correspond to expansive graph subshifts [5], in the same way that space-times of cellular automata correspond to expansive tilings. One of the difficulties in establishing such a result is that the naming conventions we adopt for our vertices systematically prevent our space-times from being cyclic, even when the dynamics described is periodic—whereas the corresponding graph subshift will in fact be cyclic.

*Acknowledgements* We wish to thank Nicolas Behr for helpful discussions. This project was partially funded by the European Union through the MSCA SE project QCOMICAL (Grant Agreement ID: 101182520), by the French National Research Agency (ANR): projects TaQC ANR-22-CE47-0012 and within the framework of 'Plan France 2030', under the research projects EPIQ ANR-22-PETQ-0007, OQULUS ANR-23-PETQ-0013, HQI-Acquisition ANR-22-PNCQ-0001 and HQI-R&D ANR-22-PNCQ-0002, and by the WOST, WithOut SpaceTime project (https://withoutspacetime.org), grant ID# 63683 from the John Templeton Foundation (JTF). The opinions expressed in this work are those of the author(s) and do not necessarily reflect the views of the JTF.

## References

1. Arrighi, P., Dowek, G.: Causal graph dynamics (long version). Information and Computation **223**, 78–93 (2013)
2. Arrighi, P., Martiel, S., Nesme, V.: Cellular automata over generalized Cayley graphs. Mathematical Structures in Computer Science **18**, 340–383 (2018), pre-print arXiv:1212.0027
3. Arrighi, P., Martiel, S., Perdrix, S.: Block representation of reversible causal graph dynamics. In: FCT 2015. pp. 351–363. Springer (2015)
4. Arrighi, P., Christodoulou, M., Durbec, A.: On quantum superpositions of graphs. arXiv preprint arXiv:2010.13579 (2020)
5. Arrighi, P., Durbec, A., Guillon, P.: Graph subshifts. In: Vedova, G.D., Dundua, B., Lempp, S., Manea, F. (eds.) Computability in Europe (CiE 2023). Lecture Notes in Computer Science, vol. 13967, pp. 261–274. Springer (2023). DOI 10.1007/978-3-031-36978-0_21
6. Arrighi, P., Martiel, S., Perdrix, S.: Reversible causal graph dynamics: invertibility, block representation, vertex-preservation. Natural Computing **19**(1), 157–178 (2020), pre-print arXiv:1502.04368
7. Baldan, P., Corradini, A., Heindel, T., König, B., Sobociński, P.: Unfolding grammars in adhesive categories. In: Kurz, A., Lenisa, M., Tarlecki, A. (eds.) CALCO 2009. pp. 350–366. Springer (2009)
8. Behr, N., Harmer, R., Krivine, J.: Fundamentals of compositional rewriting theory. Journal of Logical and Algebraic Methods in Programming **135**, 100893 (2023)
9. Bombelli, L., Lee, J., Meyer, D., Sorkin, R.D.: Space-time as a causal set. Physical review letters **59**(5), 521 (1987)
10. Bonchi, F., Gadducci, F., Kissinger, A., Sobociński, P., Zanasi, F.: Confluence of graph rewriting with interfaces. In: Yang, H. (ed.) ESOP 2017. pp. 141–169. Springer (2017)
11. Boutillier, P., Camporesi, F., Coquet, J., Feret, J., Lỳ, K.Q., Théret, N., Vignet, P.: Kasa: A static analyzer for kappa. In: CMSB 2018. pp. 285–291. Springer (2018)

12. Chalopin, J., Das, S., Widmayer, P.: Deterministic symmetric rendezvous in arbitrary graphs: overcoming anonymity, failures and uncertainty. In: Search Theory, pp. 175–195. Springer (2013)
13. Chatain, T., Haar, S., Paulevé, L.: Boolean networks: Beyond generalized asynchronicity. In: Baetens, J.M., Kutrib, M. (eds.) AUTOMATA 2018. pp. 29–42. Springer (2018)
14. Ehrig, H., Ehrig, K., Prange, U., Taentzer, G.: Fundamentals of algebraic graph transformation. Springer-Verlag New York Inc (2006)
15. Ehrig, H., Kreowski, H.J.: Parallelism of manipulations in multidimensional information structures. In: Mazurkiewicz, A. (ed.) MFCS 1976. pp. 284–293. Springer (1976)
16. Ermel, C., Rudolf, M., Taentzer, G.: The agg approach: Language and environment. In: Handbook of Graph Grammars, vol. 2, pp. 551–603. World Scientific (1999)
17. Fatès, N., Éric Thierry, Morvan, M., Schabanel, N.: Fully asynchronous behavior of double-quiescent elementary cellular automata. Theoretical Computer Science **362**(1), 1–16 (2006). DOI 10.1016/j.tcs.2006.05.036
18. Gács, P.: Deterministic computations whose history is independent of the order of asynchronous updating. arXiv preprint cs/0101026 (2001)
19. Gorard, J.: Some relativistic and gravitational properties of the wolfram model. arXiv preprint arXiv:2004.14810 (2020)
20. Hristakiev, I., Plump, D.: Checking graph programs for confluence. In: STAF 2017 Collocated Workshops. pp. 92–108. Springer (2018)
21. Ibarra, O.H., Jiang, T.: On the computing power of one-way cellular arrays. In: ICALP 1987. p. 550562. Springer, London, UK. (1987)
22. Jouannaud, J.P., Orejas, F.: Unification of drags and confluence of drag rewriting. Journal of Logical and Algebraic Methods in Programming **131**, 100845 (2023). DOI 10.1016/j.jlamp.2022.100845
23. Lambers, L., Ehrig, H., Orejas, F.: Efficient conflict detection in graph transformation systems by essential critical pairs. Electronic Notes in Theoretical Computer Science **211**, 17–26 (2008). DOI 10.1016/j.entcs.2008.04.026, GT-VMT 2006
24. Levy, J.J.: An algebraic interpretation of the $\lambda\beta$k-calculus and a labelled $\lambda$-calculus. In: Böhm, C. (ed.) $\lambda$-Calculus and Computer Science Theory. pp. 147–165. Springer (1975)
25. Lumer, E.D., Nicolis, G.: Synchronous versus asynchronous dynamics in spatially distributed systems. Physica D: Nonlinear Phenomena **71**(4), 440–452 (1994)
26. Maignan, L., Spicher, A.: Global graph transformations. In: Plump, D. (ed.) GCM 2015. CEUR, vol. 1403, pp. 34–49 (2015)
27. Mairesse, J., Marcovici, I.: Around probabilistic cellular automata. Theoretical Computer Science **559**, 42–72 (2014). DOI 10.1016/j.tcs.2014.09.009
28. Misner, C.W., Thorne, K.S., Wheeler, J.A.: Gravitation. W. H. Freeman and Company (1973)
29. Nehaniv, C.L.: Asynchronous automata networks can emulate any synchronous automata network. International Journal of Algebra and Computation **14**(05n06), 719–739 (2004)
30. Nishimura, N.: Efficient asynchronous simulation of a class of synchronous parallel algorithms. Journal of Computer and System Sciences **50**(1), 98–113 (1995)
31. Noual, M., Sené, S.: Synchronism versus asynchronism in monotonic boolean automata networks. Natural Computing **17**, 393–402 (2018)
32. Ölveczky, P.C., Meseguer, J.: The real-time maude tool. In: TACAS 2008. pp. 332–336. Springer (2008)
33. van Oostrom, V.: On causal equivalence by tracing in string rewriting. In: Grabmayer, C. (ed.) TERMGRAPH@FSCD 2022. EPTCS, vol. 377, pp. 27–43 (2022). DOI 10.4204/EPTCS.377.2
34. Rozenberg, G.: Handbook of Graph Grammars: Foundations, vol. 1. World Scientific (2003)
35. Schönfisch, B., de Roos, A.: Synchronous and asynchronous updating in cellular automata. Biosystems **51**(3), 123–143 (1999). DOI 10.1016/S0303-2647(99)00025-8
36. Sorkin, R.: Time-evolution problem in Regge calculus. Phys. Rev. D. **12**(2), 385–396 (1975)